

```

1  """API Rate Limiter Service - FastAPI 진입점.
2
3  실행
4  py -muvicornmainapp--reload
5
6  Redis가 없으면 백엔드 백을 사용하지한다
7  문서http://127.0.1:8000/docs
8  """
9  from__future__importannotations
10
11  importlogging
12  fromcontextlibimportasynccontextmanager
13
14  fromfastapiimportDependsFastAPI,Request
15  fromfastapi.openapi.docsimportget_swagger_ui_html
16  fromfastapi.responsesimportHTMLResponse,JSONResponse
17
18  fromappapiimportrate_limitrouter
19  fromappconfigimportsettings
20  fromapplimiteimportRateLimitExceeded,daild_limiter
21  fromappnotifierimportNotifier
22
23  loginbasicConf(level=logging.INFO)
24  logger= logging.getLogger("ratelimiter")
25
26
27  @asynccontextmanager
28  asyncdeflifespan(app: FastAPI)
29      appstatelimiter= awaitbuild_limiter(settings.redis_url)
30      appstatenotifier= Notifier(settings)
31      loggerinfo("Ratelimitebackend %s", type(appstatelimiter).__name__)
32      loggerinfo(
33          "Notifier          : enabled=%schannel=%s"
34          appstatenotifierenabled
35          settingotify_chanel
36      )
37      try:
38          yield
39      finally:
40          awaitappstatelimiter.close()
41
42
43  # Swagger UI 문서 화면의 태그(섹션) 순서 및 한국어 설명
44  _TAGS_METADATA=
45      {
46          "name": "요청한" , "description": "요청 용량과 현재 남은 용량, 핸드폰인트"
47          "name": "시스템" , "description": "서버상에서 총 백엔드 인트"
48          "name": "예시" , "description": "ratelimit에 용량과 유효"
49      }
50
51  app= FastAPI(
52      title="API Rate Limiter 서비스"
53      version="1.0.0",
54      description=
55          """
56          클라이언트 요청을 요청한에 따라 제한하는
57          백엔드 시스템이다
58          - **알고리즘** Bucket(버킷)을 평 균 분 배 하
59          - **백엔드** Redis가 없으면 백엔드 백을 사용하지한다
60          - **클라이언트** X-API-Key 헤더가 없으면 P\ n\ n
61          "200 응답은 유효한 요청 내보내게 된 것을 출력해줍니다
62      ),
63      openapi_tags=_TAGS_METADATA,
64      lifespan=lifespan,
65      docs_url=None, # 기본 문서 화면을 한국어 문서 화면으로
66  )
67
68  appinclude_router(rate_limitrouter)
69
70  # --- Swagger UI 버튼/라벨을 한국어로 치환하는 스크립트 ---
71  # Swagger UI 자체는 영어 UI만 제공하므로, 렌더링된 화면의 고정 라벨 텍스트를
72  # 사전(dict) 기반으로 바꿔준다. 입력 필드(INPUT/TEXTAREA)는 건드리지 않는다.
73  _KOREAN_DOCS_SCRIPT=
74  <script>
75  (function() {
76      vardict= {
77          "Tryitout": "요청내보내기", "cancel": "취소", "Execute": "실행"
78          "Clear": "초기화", "Reset": "초기화", "Parameters": "파라미터"
79          "NoParameters": "파라미터 없음", "Name": "이름", "Description": "설명"
80          "RequestBody": "요청문", "Responses": "응답", "ResponseBody": "응답문"
81          "ServerResponse": "서버응답", "ResponseSamples": "응답예시", "Code": "코드"
82          "Details": "상세", "MediaType": "미디어 타입", "ExampleValue": "예값"
83          "Schema": "스키마", "Schemas": "스키마", "Download": "다운로드"
84          "Authorize": "인증", "AvailableAuthorizations": "사용가능한 인증"
85          "Close": "닫기", "RequestURL": "요청URL", "Loading": "로딩중", "볼러주는",
86          "required": "필수", "Links": "링크", "NoLinks": "링크 없음"
87          "SendEmptyValue": "빈값보내기", "ExpandAll": "모두 펼치기"
88          "CollapseAll": "모두 접기", "Copyto clipboard": "클립보드에 복사"
89          "SuccessfulResponse": "성공응답", "ValidationError": "검출 오류"
90          "Error": "오류", "Warning": "경고", "Deprecated": "지정된 단점"
91          "Controls": "제어사항", "header": "헤더", "Accept": "Accept"
92          "EditValue": "값수정", "Model": "모델", "Servers": "서버"
93          "Query": "쿼리", "Path": "경로", "Header": "헤더"
94          "Filterbytag": "태그 필터", "NoParameter's": "파라미터 없음"
95      };
96      functiontranslate(root) {
97          varwalker= documentcreateTreeWalker(root, NodeFilter.SHOW_TEXT);
98          varnodes= [], n;
99          while((n = walker.nextNode())) nodespush(n);
100          nodesforEach(function(node) {
101              varv= (node.nodeValue|| "").trim();
102              varparent= node.parentNode;
103              if (!parent) return;
104              if (/^(INPUT|TEXTAREA|SCRIPT|STYLE|CODE|IFRAME|pre)$/.test(parent.tagName)) return;
105              if (dict[v]) node.nodeValue= node.nodeValue.replace(/%s/g, dict[v]);
106          });
107      }
108      varobs= newMutationObserver(function() { translate(documentbody); });
109      windowaddEventListener("load", function() {
110          setTimeout(function() {
111              translate(documentbody);
112              obs.observe(documentbody { childListtrue, subtreetrue});
113          }, 300);
114      });
115  })();
116  </script>
117  """
118
119  @appget("/docs", include_in_schema=False)
120  asyncdefkorean_swagger()> HTMLResponse
121      """기본 Swagger UI에 한국어 라벨 치환 스크립트를 주입한 문서 화면."""
122      base= get_swagger_ui_html(openapi_url=app.openapi_url, title={app.title} - API 문서"
123      html= basebodydecode().replace("</body>", _KOREAN_DOCS_SCRIPT"/body>")
124      returnHTMLResponse(html)
125
126  @appexception_handler(RateLimitExceeded)
127  asyncdef_rate_limit_handler(request: Request, exc: RateLimitExceeded) > JSONResponse
128      r = exc.result
129      headers= {
130          "Retry-After": str(int(r.retry_after) + 1),
131          "X-RateLimit-Limit": str(r.limit),
132          "X-RateLimit-Remaining": str(r.remaining)
133      }
134      returnJSONResponse(
135          status_code=429
136          headers=headers
137          content=
138              {
139                  "error": "rate limit exceeded"
140                  "message": "요청 도량과 현재 남은 용량과 세 도하세요"
141                  "client_id": exc.client_id
142                  "retry_after": round(r.retry_after, 2)
143              }
144      )
145
146  @appget(
147      "/health",
148      tags=["시스템"],
149      summary="상태 확인"
150      response=200 { "description": "서버 정상", "status": "backend" },
151  )
152  asyncdefhealth(request: Request) > dict[str, object]
153      """서비스 상태 및 사용 중인 백엔드 확인."""
154      limiter= request.appstatelimiter
155      return{ "status": "ok", "backend": type(limiter).__name__ }
156
157  # --- 예시: rate limit으로 보호되는 실제 엔드포인트 ---
158  @appget(
159      "/demo",
160      tags=["예시"],
161      summary="외부 서비스 아웃"
162      response=
163          200 { "description": "요청 용량"
164              429 { "description": "요청 도량과 현재 남은 용량과 세 도하세요"
165                  "Retry-After": round(r.retry_after, 2)
166              },

```



```

1  """애플리케이션 설정 (환경 변수 기반).
2
3  .env 파일을 환경 변수로 읽어와 envexample 참고
4  """
5  from __future__ import annotations
6
7  imports
8  from dataclasses import dataclass
9
10
11 def _get_bool(name: str, default: bool) -> bool
12     val = os.getenv(name)
13     if val is None:
14         return default
15     return val.strip().lower() in {"1", "true", "yes", "on"}
16
17
18 def _get_int(name: str, default: int) -> int
19     try:
20         return int(os.getenv(name, str(default)))
21     except (TypeError, ValueError):
22         return default
23
24
25 @dataclass(frozen=True)
26 class Settings
27     # Redis 연결 URL. 비어 있으면 인메모리 백엔드로 자동 폴백한다.
28     redis_url: str | None = os.getenv("REDIS_URL") or None
29
30     # 기본 Rate Limit 정책 (Token Bucket)
31     default_requests: int = _get_int("RATE_LIMIT_REQUESTS", 100)
32     default_window: int = _get_int("RATE_LIMIT_WINDOW", 60)
33     # burst=0 이면 requests 값과 동일하게 취급 (아래 default_burst_eff
34     default_burst: int = _get_int("RATE_LIMIT_BURST", 0)
35
36     # 장애 시 정책: True면 통과(fail-open), False면 차단(fail-close)
37     fail_open: bool = _get_bool("FAIL_OPEN", True)
38
39     # 알림 채널: none | slack | discord | telegram
40     notify_channel: str = (os.getenv("NOTIFY_CHANNEL") or "none").lower()
41     notify_webhook: str | None = os.getenv("NOTIFY_WEBHOOK_URL") or None
42     telegram_bot_token: str | None = os.getenv("TELEGRAM_BOT_TOKEN") or None
43     telegram_chat_id: str | None = os.getenv("TELEGRAM_CHAT_ID") or None
44     # 동일 이벤트 중복 알림 방지용 쿨다운(초)
45     notify_cooldown: int = _get_int("NOTIFY_COOLDOWN", 300)
46
47     @property
48     def default_burst_effective(self) -> int
49         return self.default_burst * self.default_requests
50
51
52 settings: Settings

```



```

1  """요청/응답 및 정책 데이터 모델 (Pydantic)."""
2  from __future__ import annotations
3
4  from pydantic import BaseModel, Field
5
6
7  class CheckRequest(BaseModel):
8      client_id: str | None = Field(
9          None, description="클라이언트 ID (없으면 기본 요청)"
10     )
11     endpoint: str = Field("default", description="엔드포인트 (기본 리소스)")
12     cost: int = Field(1, ge=1, description="요청 비용")
13
14
15  class CheckResponse(BaseModel):
16      allowed: bool
17      client_id: str
18      limit: int
19      remaining: int
20      reset_after: float = Field(..., description="메트릭이 만료될 때")
21      retry_after: float = Field(0, description="요청을 재시도할 때 (허용 0)")
22
23
24  class UsageResponse(BaseModel):
25      client_id: str
26      limit: int
27      remaining: int
28      reset_after: float
29
30
31  class PolicyModel(BaseModel):
32      """Token Bucket 파라미터로 환산되는 제한 정책."""
33
34      requests: int = Field(..., ge=1, description="초당 요청 수")
35      window_seconds: int = Field(..., ge=1, description="창(초)")
36      burst: int = Field(0, ge=0, description="버스트 크기 (requests 창에)")
37
38      @property
39      def capacity(self) -> int:
40          """버킷 용량 = 최대로 몰아 쓸 수 있는 토큰 수."""
41          return self.burst + self.requests
42
43      @property
44      def refill_rate(self) -> float:
45          """초당 토큰 충전 속도."""
46          return self.requests / self.window_seconds
47
48
49  class PolicyUpsert(BaseModel):
50      plan: str = Field(..., description="요구 사항 (e.g., 'prg enterprise.')")
51      policy: PolicyModel

```

```

1  """요금제(plan)별 Rate Limit 정책 저장소.
2
3  스케일러블메시나 클라우드 환경에서 Redis/DB에서
4  로드-저장해결을 API(PUT/v1/policy)를동작시킨다
5  """
6  from __future__ import annotations
7
8  from config import settings
9  from models import PolicyModel
10
11
12 class PolicyStore
13     def __init__(self) -> None
14         self.default = PolicyModel
15         requests = settings.default_requests
16         window_seconds = settings.default_window
17         burst = settings.default_burst_effective
18     )
19     self.policies = dict[str, PolicyModel] {
20         "free": self.default
21         "pro": PolicyModel(requests=1000, window_seconds=60, burst=120)
22         "enterprise": PolicyModel(requests=10000, window_seconds=60, burst=1500)
23     }
24
25     def get(self, plan: str) -> PolicyModel
26         return self.policies.get(plan, self.default)
27
28     def upsert(self, plan: str, policy: PolicyModel) -> None
29         self.policies[plan] = policy
30
31     def all(self) -> dict[str, PolicyModel]
32         return dict(self.policies)
33
34
35 policy_store = PolicyStore()

```



```

1  """클라이언트 식별 로직.
2
3  우선순위에 따라 client_id, X-API-Key 헤더 순으로
4  반환함(client_id plan)
5  """
6  from __future__ import annotations
7
8  from fastapi import Request
9
10 # 데모용 API Key -> 요금제 매핑. 실제로는 DB/Redis에서 조회한다.
11 API_KEYS = {str: str} = {
12     "demo-free-key": "free",
13     "demo-pro-key": "pro",
14     "demo-enterprise-key": "enterprise"
15 }
16
17
18 def identify_client(request: Request, explicit_id: str | None = None) -> tuple[str, str]:
19     api_key = request.headers.get("X-API-Key")
20     if api_key and api_key in API_KEYS:
21         plan = API_KEYS[api_key]
22         return (explicit_id if explicit_id else f"key{api_key}", plan)
23
24     # API Key가 없으면 IP로 폴백 (free 정책 적용)
25     client_host = request.client.host if request.client else "unknown"
26     return (explicit_id if explicit_id else f"ip{client_host}", "free")

```



```

1  """Rate Limiter - Token Bucket 알고리즘.
2
3  두가채제공한다
4  - InMemoryTokenBucket: 로컬로 세팅을 미시할땐
5  - RedisTokenBucket: Redis Lua 스크립트를 사용(분산)
6
7  build_limiter()가 설정에 따라 적절한 선택을 해 Redis 연결
8  실패시 자동으로 폴백한다
9  """
10 from __future__ import annotations
11
12 import asyncio
13 import logging
14 import time
15 from dataclasses import dataclass
16
17 from model import PolicyModel
18
19 logger = logging.getLogger(f"ratelimiter:limiter")
20
21
22 @dataclass
23 class LimitResult
24     allowed: bool
25     limit: int
26     remaining: int
27     reset_after: float # 버킷이 차기까지
28     retry_after: float # 다음청때까지(허용)
29
30
31 class RateLimitException(Exception)
32     """제한 초과 시 발생. main의 예외 핸들러가 429로 변환한다."""
33
34     def __init__(self, result: "LimitResult", client_id: str) -> None
35         self.result = result
36         self.client_id = client_id
37         super().__init__(f"rate limit exceeded for {client_id}")
38
39
40 def _derive(allowed: bool, tokens: float, policy: PolicyModel, cost: int) -> LimitResult
41     refill = policy.refill_rate
42     capacity = policy.capacity
43     remaining = int(tokens)
44     reset_after = (capacity - tokens) / refill if refill > 0 else 0.0
45     if allowed:
46         retry_after = 0.0
47     else:
48         retry_after = (cost - tokens) / refill if refill > 0 else 0.0
49     return LimitResult(allowed, capacity, remaining, reset_after, retry_after)
50
51
52 class RateLimiter
53     """Rate Limiter 인터페이스."""
54
55     async def check(self, key: str, policy: PolicyModel, cost: int = 1) -> LimitResult
56         raise NotImplementedError
57
58     async def peek(self, key: str, policy: PolicyModel) -> LimitResult
59         """토큰 소비 없이 현재 잔여량만 조회 (cost=0)."""
60         return await self.check(key, policy, cost=0)
61
62     async def close(self) -> None # pragma: no cover
63         pass
64
65
66 class InMemoryTokenBucket(RateLimiter)
67     def __init__(self) -> None
68         # key -> (tokens, last_monotonic_ts)
69         self.state: dict[str, tuple[float, float]] = {}
70         self.lock = asyncio.Lock()
71
72     async def check(self, key: str, policy: PolicyModel, cost: int = 1) -> LimitResult
73         capacity = policy.capacity
74         refill = policy.refill_rate
75         now = time.monotonic()
76         async with self.lock:
77             tokens, last = self.state.get(key, (float(capacity), now))
78             tokens = min(capacity, tokens + (now - last) * refill)
79             allowed = tokens >= cost
80             if allowed:
81                 tokens -= cost
82             self.state[key] = (tokens, now)
83         return _derive(allowed, tokens, policy, cost)
84
85
86 # 원자적 토큰 버킷 (KEYS[1]=버킷 키, ARGV=capacity, refill, now, cost)
87 _TOKEN_BUCKET_LUA = """
88 local key = KEYS[1]
89 local capacity = tonumber(ARGV[1])
90 local refill = tonumber(ARGV[2])
91 local now = tonumber(ARGV[3])
92 local cost = tonumber(ARGV[4])
93 local data = redis.call('HMGET', key, 'tokens', 'ts')
94 local tokens = tonumber(data[1])
95 local ts = tonumber(data[2])
96 if tokens == nil then
97     tokens = capacity
98     ts = now
99 end
100 local delta = math.max(0, now - ts)
101 tokens = math.min(capacity, tokens + delta * refill)
102 local allowed = 0
103 if tokens >= cost then
104     tokens = tokens - cost
105     allowed = 1
106 end
107 redis.call('HSET', key, 'tokens', tokens, 'ts', now)
108 local ttl = math.ceil(capacity / refill) + 1
109 redis.call('EXPIRE', key, ttl)
110 return {allowed, tostring(tokens)}
111 """
112
113
114 class RedisTokenBucket(RateLimiter)
115     def __init__(self, client) -> None # client: redis.asyncio.Redis
116         self.redis = client
117         self.script = client.register_script(_TOKEN_BUCKET_LUA)
118
119     async def check(self, key: str, policy: PolicyModel, cost: int = 1) -> LimitResult
120         now = time.time()
121         allowed, tokens_raw = await self.script(
122             keys=[f"rl:{key}"],
123             args=[policy.capacity, policy.refill_rate, now, cost],
124         )
125         tokens = float(tokens_raw)
126         allowed = int(allowed_raw) == 1
127         return _derive(allowed, tokens, policy, cost)
128
129     async def close(self) -> None
130         await self.redis.close()
131
132
133 async def build_limiter(redis_url: str) -> RateLimiter
134     """설정 따라 백엔드를 생성. Redis 실패 시 인메모리로 폴백한다."""
135     if not redis_url:
136         return InMemoryTokenBucket()
137     try:
138         import redis.asyncio as aioredis # 지원 포트를 Redis로 실행
139
140         client = aioredis.from_url(redis_url, encoding="utf-8", decode_responses=True)
141         await client.ping()
142         logger.info(f"Redis 백엔드 생성 성공", redis_url)
143         return RedisTokenBucket(client)
144     except Exception as exc: # noqa BLE001
145         logger.warning(f"Redis 연결(실패) - 인메모리 백엔드로 폴백함", exc)
146         return InMemoryTokenBucket()

```



```

1  """알림(Notification) 연동.
2
3  이 모듈은 생체 인식 센서와 Slack/Discord/Telegram Webhook을 통해
4  전송할 수 있는 이벤트에 대해 사소한 동작을 한번 처리한다
5
6  - 쿨다운을 이벤트 발생 횟수를 디바운스
7  - 전송 패킷은 크기를 잘 조정하여 전송한다
8  """
9  from __future__ import annotations
10
11  import logging
12  import time
13  from typing import Any
14
15  from config import Settings
16
17  logger = logging.getLogger("ratelimit_notifier")
18
19
20  class Notifier
21      def __init__(self, settings: Settings) -> None
22          self.channel = settings.notify_channel
23          self.webhook_url = settings.notify_webhook_url
24          self.tg_token = settings.telegram_bot_token
25          self.tg_chat = settings.telegram_chat_id
26          self.cooldown = settings.notify_cooldown
27          self.last_sent: dict[str, float] = {} # event_key > last monotonic time
28
29      @property
30      def enabled(self) -> bool
31          if self.channel == "telegram"
32              return bool(self.tg_token and self.tg_chat)
33          if self.channel in {"slack", "discord"}:
34              return bool(self.webhook_url)
35          return False
36
37      async def notify(
38          self,
39          event_key: str,
40          title: str,
41          fields: dict[str, Any],
42          severity: str = "warning"
43      ) -> bool
44          """알림 전송. 쿨다운 내 동일 event_key면 skip하고 False 반환."""
45          if not self.enabled:
46              return False
47
48          now = time.monotonic()
49          last = self.last_sent.get(event_key)
50          if last is not None and (now - last) < self.cooldown:
51              return False
52          self.last_sent[event_key] = now
53
54          try:
55              await self._send(title, fields, severity)
56              return True
57          except Exception as exc: # noqa BLE001
58              # 실패 시 재시도/로컬 백업 로직을 여기에 확장한다 (스켈레톤은)
59              logger.warning("알림 전송 실패", exc)
60              return False
61
62      async def _send(self, title: str, fields: dict[str, Any], severity: str) -> None
63          import httpx # 시스템 포트로 httpx 미설치 시 에러
64
65          lines = [f"[{severity.upper()}] {title}"]
66          lines += [f"- {k}: {v}" for k, v in fields.items()]
67          text = "\n".join(lines)
68
69          async with httpx.AsyncClient(timeout=5) as http
70              if self.channel == "slack":
71                  await http.post(self.webhook_url, json={"text": text})
72              elif self.channel == "discord":
73                  await http.post(self.webhook_url, json={"content": text})
74              elif self.channel == "telegram":
75                  url = f"http://api.telegram.org/bot{self.tg_token}/sendMessage"
76                  await http.post(url, json={"chat_id": self.tg_chat, "text": text})

```



```

1  """API 라우터 및 재사용 가능한 rate limit 의존성.
2
3  엔드포인트별 제한 기준
4  POST /v1/check          요청 용량 제한
5  GET /v1/usage/{client_id} 사용량 조회
6  GET /v1/policies        정책 조회
7  PUT /v1/policies        정책 등록/수정
8  """
9  from __future__ import annotations
10
11  import logging
12
13  from fastapi import APIRouter, Depends, Request
14
15  from config import settings
16  from identify import identify_client
17  from limiter import LimitResult, RateLimiter, RateLimitExceeded
18  from model import (
19      CheckRequest,
20      CheckResponse,
21      PolicyModel,
22      PolicyUpsert,
23      UsageResponse,
24  )
25  from notifier import Notifier
26  from policies import policy_store
27
28  logger = logging.getLogger("ratelimitapi")
29
30  router = APIRouter(prefix="/v1", tags=["요청 제한"])
31
32
33  def get_limit(request: Request) -> RateLimiter:
34      return request.app.state.limiter
35
36
37  def get_notifier(request: Request) -> Notifier:
38      return request.app.state.notifier
39
40
41  async def evaluate(
42      request: Request,
43      explicit_id: str | None,
44      endpoint: str,
45      cost: int,
46      limiter: RateLimiter,
47      notifier: Notifier | None,
48  ) -> tuple[str, str, LimitResult]:
49      """식별 → 정책 조회 → 제한 검사 → (초과 시) 알림."""
50      client_id, plan = identify_client(request, explicit_id)
51      policy = policy_store.get(plan)
52
53      try:
54          result = await limiter.check(f"{client_id}{endpoint}", policy, cost=cost)
55      except Exception as exc: # noqa BLE001 백엔드에서 fail-open/close 정책 적용
56          logger.warning("limiter 백엔드(%s) - fail_open=%s", exc, settings.fail_open)
57          if settings.fail_open:
58              return client_id, plan, LimitResult(True, policy.capacity, policy.capacity * 0.0, 0.0)
59          return client_id, plan, LimitResult(False, policy.capacity * 0.0, 0.0, 1.0)
60
61      if not result.allowed and notifier is not None:
62          await notifier.notify(
63              event_key=f"exceed_{client_id}",
64              title="Rate limit 초과 시",
65              fields={
66                  "client": client_id,
67                  "plan": plan,
68                  "endpoint": endpoint,
69                  "limit": result.limit,
70                  "retry_after(s)": round(result.retry_after, 2),
71              },
72              severity="warning"
73          )
74      return client_id, plan, result
75
76
77  @router.post("/check")
78      response_model=CheckResponse
79      summary="요청 용량 제한"
80      response =
81          200 {"description": "판정 후 allowed remaining retry_after 반환"},
82          422 {"description": "요청 식별에 cost가 미흡"},
83      },
84  )
85
86  async def check(
87      body: CheckRequest,
88      request: Request,
89      limiter: RateLimiter = Depends(get_limit),
90      notifier: Notifier = Depends(get_notifier),
91  ) -> CheckResponse:
92      client_id, plan, result = await evaluate(
93          request, body.client_id, body.endpoint, body.cost, limiter, notifier
94      )
95      return CheckResponse(
96          allowed=result.allowed,
97          client_id=client_id,
98          limit=result.limit,
99          remaining=result.remaining,
100         reset_after=round(result.reset_after, 2),
101         retry_after=round(result.retry_after, 2),
102     )
103
104
105  @router.get("/usage/{client_id}")
106      response_model=UsageResponse
107      summary="사용량 조회"
108      response =
109          200 {"description": "조회된 사용량 반환"},
110          422 {"description": "요청 식별 실패"},
111      },
112  )
113
114  async def usage(
115      client_id: str,
116      request: Request,
117      endpoint: str = "default",
118      limiter: RateLimiter = Depends(get_limit),
119  ) -> UsageResponse:
120      _client_id, plan = identify_client(request, client_id)
121      policy = policy_store.get(plan)
122      result = await limiter.peek(f"{client_id}{endpoint}", policy)
123      return UsageResponse(
124          client_id=client_id,
125          limit=result.limit,
126          remaining=result.remaining,
127          reset_after=round(result.reset_after, 2),
128      )
129
130
131  @router.get("/policies")
132      response_model=list[PolicyModel]
133      summary="정책 조회"
134      response =
135          200 {"description": "요청된 정책 목록"},
136      )
137  async def list_policies() -> list[str, PolicyModel]:
138      return policy_store.all()
139
140
141  @router.put("/policies")
142      summary="정책 등록/수정"
143      response =
144          200 {"description": "등록 성공"},
145          422 {"description": "요청 식별 실패"},
146      },
147  )
148
149  async def upsert_policy(body: PolicyUpsert) -> dict[str, str]:
150      policy_store.upsert(body.plan, body.policy)
151      return {"status": "ok", "plan": body.plan}
152
153
154  def rate_limit(endpoint: str = "default", cost: int = 1):
155      """임의의 라우터를 보호하는 의존성 팩토리.
156
157      사용
158      @app.get("/demo")
159      async def demo():
160          Depends(rate_limit(endpoint="demo"))
161      """
162
163
164  async def depend(request: Request) -> LimitResult:
165      limiter = RateLimiter(request.app.state.limiter)
166      notifier = Notifier(request.app.state.notifier)
167      _client_id, plan, result = await evaluate(request, None, endpoint, cost, limiter, notifier)
168      if not result.allowed:
169          raise RateLimitExceeded(result.reset_after, _client_id)
170      return result
171
172  return dependency
```



```

1  """Token Bucket 핵심 로직 단위 테스트.
2
3  pytest-asyncio를 이용해 async 코루틴을 asyncio.run()으로 실행한다
4  """
5  from __future__ import annotations
6
7  import asyncio
8  import time
9
10 from applimiter import InMemoryTokenBucket, derive
11 from appmodels import PolicyModel
12
13
14 def run(cor):
15     return asyncio.run(cor)
16
17
18 # — PolicyModel 파생 값 —————
19 def test_policy_capacity_defaults_to(requests):
20     p = PolicyModel(requests=100, window_seconds=60, burst=0)
21     assert p.capacity == 100 # burst=0 requests
22     assert p.refill_rate == 100 / 60
23
24
25 def test_policy_capacity_uses_burst_when_set():
26     p = PolicyModel(requests=100, window_seconds=60, burst=1200)
27     assert p.capacity == 1200
28     assert p.refill_rate == 1000 / 60
29
30
31 # — _derive: allowed/remaining/retry_after 계산 —————
32 def test_derive_blocked_sets_positive_retry_after():
33     p = PolicyModel(requests=3, window_seconds=60, burst=3) # refill 1 token/s
34     r = _derive(allowed=False, tokens=0, policy=p, cost=1)
35     assert r.allowed == False
36     assert r.remaining == 0
37     assert r.retry_after == 1.0 # (cost- tokens) / refill == 1 / 1
38
39     r_ok = _derive(allowed=True, tokens=0, policy=p, cost=1)
40     assert r_ok.retry_after == 0.0 # 허용항상
41
42
43 # — InMemoryTokenBucket: 허용 → 소진 → 차단 —————
44 def test_bucket_allows_up_to_capacity_then_blocks():
45     p = PolicyModel(requests=3, window_seconds=60, burst=3) # 리플렛우림
46     bucket = InMemoryTokenBucket()
47     key = "client_endpoint"
48
49     results = [run(bucket.check(key, p)) for _ in range(3)]
50     assert all(r.allowed for r in results)
51     assert results[0].remaining == 2
52     assert results[1].remaining == 1
53     assert results[2].remaining == 0
54
55     blocked = run(bucket.check(key, p))
56     assert blocked.allowed == False
57     assert blocked.remaining == 0
58     assert blocked.retry_after == 0
59     assert blocked.limit == 3
60
61
62 # — 시간 경과 시 토큰 리필 —————
63 def test_bucket_refills_over_time():
64     # capacity 3, window 1s → refill 3 tokens/s. 0.5s면 1개 이상 회복
65     p = PolicyModel(requests=3, window_seconds=60, burst=3)
66     bucket = InMemoryTokenBucket()
67     key = "client_endpoint"
68
69     for _ in range(3):
70         run(bucket.check(key, p))
71         assert run(bucket.check(key, p)).allowed == False # 소진 차단
72
73     time.sleep(0.5)
74     assert run(bucket.check(key, p)).allowed == True # 리플렛재허용
75
76
77 # — peek(cost=0): 소비 없이 조회 —————
78 def test_peek_does_not_consume_tokens():
79     p = PolicyModel(requests=10, window_seconds=60, burst=10)
80     bucket = InMemoryTokenBucket()
81
82     first = run(bucket.peek("key", p))
83     second = run(bucket.peek("key", p))
84     assert first.allowed == True
85     assert first.remaining == 10
86     assert second.remaining == 10 # 두번peek해도대로
87
88
89 # — cost가 잔여 토큰보다 크면 차단 —————
90 def test_cost_greater_than_available_blocks():
91     p = PolicyModel(requests=5, window_seconds=60, burst=5)
92     bucket = InMemoryTokenBucket()
93     r = run(bucket.check("key", p, cost=6))
94     assert r.allowed == False
95     assert r.remaining == 5 # 소비함
96
97
98 # — 서로 다른 key는 독립적인 버킷 —————
99 def test_keys_are_isolated():
100     p = PolicyModel(requests=1, window_seconds=60, burst=1)
101     bucket = InMemoryTokenBucket()
102     assert run(bucket.check("a:ep", p)).allowed == True
103     assert run(bucket.check("a:ep", p)).allowed == False # a 소진
104     assert run(bucket.check("b:ep", p)).allowed == True # b는영했음

```



```

1  """클라이언트 식별 로직(identify_client) 단위 테스트.
2
3  identify_client는 request.headers.get(...)과 request.client.host만 사용하므로
4  가벼운 SimpleNamespace과 dict를 청송로 포함했다
5  """
6  from __future__ import annotations
7
8  from types import SimpleNamespace
9
10 from app.identify import identify_client
11
12
13 def make_request(headers: dict | None = None, host: str | None = "1.2.3.4"):
14     return SimpleNamespace(
15         headers=headers,
16         client=SimpleNamespace(host=host) if host is not None else None
17     )
18
19
20 def test_known_api_key_maps_to_plan():
21     cid, plan = identify_client(make_request({"X-API-Key": "demo-pro-key"}))
22     assert plan == "pro"
23     assert cid == "keydemo-pro-key"
24
25
26 def test_unknown_api_key_falls_back_to_ip_free():
27     cid, plan = identify_client(make_request({"X-API-Key": "nope"}, host="9.9.9"))
28     assert plan == "free"
29     assert cid == "ip9.9.9.9"
30
31
32 def test_no_key_uses_source_ip():
33     cid, plan = identify_client(make_request(host="5.5.5"))
34     assert (cid, plan) == ("ip5.5.5.5", "free")
35
36
37 def test_missing_client_host_becomes_unknown():
38     cid, plan = identify_client(make_request(host=None))
39     assert cid == "ipunknown"
40     assert plan == "free"
41
42
43 def test_explicit_id_overrides_but_keeps_plan():
44     cid, plan = identify_client(
45         make_request({"X-API-Key": "demo-enterprise"}, key="explicit_id="tenant-42")
46     )
47     assert cid == "tenant-42"
48     assert plan == "enterprise"

```



```

1  """Notifier(웹훅 알림) 단위 테스트.
2
3  실제 트윗을 send하는 루틴이 disabled 상태이고,
4  쿨다운이나 중복을 검증한다
5  """
6  from __future__ import annotations
7
8  import asyncio
9  import dataclasses
10
11  from appconfig import settings
12  from appnotifier import Notifier
13
14
15  def run(cor):
16      return asyncio.run(cor)
17
18
19  def settings(**overrides):
20      """기본 settings에서 일부 필드만 바꾼 frozen dataclass 사본."""
21      return dataclasses.replace(settings, **overrides)
22
23
24  def test_disabled_when_channel_is_none():
25      n = Notifier(settings.notify_channel="")
26      assert n.enabled is False
27      assert run(n.notify("e", "t", {})) is False # 비활성화
28
29
30  def test_slack_requires_webhook_url():
31      assert Notifier(settings.notify_channel="slack", notify_webhook_url=None).enabled is False
32      assert Notifier(settings.notify_channel="slack", notify_webhook_url="http://").enabled is True
33
34
35  def test_telegram_requires_token_and_chat_id():
36      assert Notifier(
37          settings.notify_channel="telegram", telegram_bot_token="", telegram_chat_id=None
38      ).enabled is False
39      assert Notifier(
40          settings.notify_channel="telegram", telegram_bot_token="", telegram_chat_id="c"
41      ).enabled is True
42
43
44  def test_cooldown_debounces_same_event_key():
45      n = Notifier(
46          settings.notify_channel="slack", notify_webhook_url="http://", notify_cooldown=300
47      )
48      sent_list = []
49
50      async def fake_send(title, fields, severity):
51          sent_list.append(title)
52
53      n._send = fake_send # 네트워크 대체
54
55      assert run(n.notify("exceed1", "first", {})) is True
56      assert run(n.notify("exceed1", "dup", {})) is False # 쿨다운 종료 → skip
57      assert run(n.notify("exceed2", "other", {})) is True # 다른 → 전송
58      assert sent_list == ["first", "other"]
59
60
61  def test_send_failure_is_swallowed():
62      n = Notifier(settings.notify_channel="slack", notify_webhook_url="http://")
63
64      async def boom(*args, **kwargs):
65          raise RuntimeError("network down")
66
67      n._send = boom
68      # 전송이 예외를 던져도 서비스 흐름을 막지 않고 False 반환
69      assert run(n.notify("k", "t", {})) is False

```



```

1  """HTTP 엔드포인트 통합 테스트 (FastAPI TestClient).
2
3  TestClient를 with 컨텍스트로써 fastapi.startup()을 실행한 appstate에
4  InMemoryTokenBucket을 limiter로 지정된 REDIS_URL이 설정된 메모리
5
6  주의 policy_store로 쓸 영어 단어를 테스트할 때 사용한다
7  매 테스트마다 리셋된다.
8  """
9  from __future__ import annotations
10
11  import pytest
12  from fastapi.testclient import TestClient
13
14  import main
15  from app.policies import policy_store
16
17
18  @pytest.fixture()
19  def client():
20      # with 컨텍스트가 lifespan을 돌려 매 테스트마다 새 limiter(빈 버킷)를 만든다
21      with TestClient(main.app) as c:
22          yield c
23
24
25  @pytest.fixture(autouse=True)
26  def reset_policy_store():
27      snapshot = policy_store.get() # dict 사본
28      yield
29      policy_store.policies = snapshot # PUT으로 변경
30
31
32  def test_health_reports_inmemory(client):
33      r = client.get("/health")
34      assert r.status_code == 200
35      body = r.json()
36      assert body["status"] == "ok"
37      assert body["backend"] == "InMemoryTokenBucket"
38
39
40  def test_list_policies_contains_defaults(client):
41      data = client.get("/v1/policies").json()
42      assert {"free", "pro", "enterprise"} == set(data)
43      assert data["pro"]["burst"] == 1200
44      assert data["enterprise"]["requests"] == 10000
45
46
47  def test_upsert_policy_adds_new_policy(client):
48      payload = {"plan": "gold", "policy": {"requests": 5, "window_seconds": 10, "burst": 8}}
49      r = client.put("/v1/policies", json=payload)
50      assert r.status_code == 200
51      assert r.json() == {"status": "ok", "plan": "gold"}
52
53      data = client.get("/v1/policies").json()
54      assert data["gold"]["burst"] == 8
55
56
57  def test_check_with_pro_key_consumes_quota(client):
58      r = client.post(
59          "/v1/check", headers={"X-API-Key": "demo-pro-key"}, json={"endpoint": "search"}
60      )
61      assert r.status_code == 200
62      body = r.json()
63      assert body["allowed"] is True
64      assert body["client_id"] == "keydemo-pro-key"
65      assert body["limit"] == 1200
66      assert body["remaining"] == 1199 # 1개 소비
67
68
69  def test_usage_peek_does_not_consume(client):
70      first = client.get("/v1/usage/tenant").json()
71      second = client.get("/v1/usage/tenant").json()
72      assert first["remaining"] == second["remaining"] # peek → 소모 없음
73      assert first["limit"] == second["remaining"] # 처음엔 남은
74
75
76  def test_demo_returns_429_with_headers_and_client_limit(client):
77      # 기본 free 정책(100)은 429까지 100회 필요 → 테스트용으로 free를 축소.
78      client.put(
79          "/v1/policies",
80          json={"plan": "free", "policy": {"requests": 2, "window_seconds": 60, "burst": 2}},
81      )
82      assert client.get("/demo").status_code == 200
83      assert client.get("/demo").status_code == 200
84
85      blocked = client.get("/demo")
86      assert blocked.status_code == 429
87      assert "Retry-After" in blocked.headers
88      assert blocked.header["X-RateLimit-Limit"] == "2"
89      assert blocked.header["X-RateLimit-Remaining"] == "0"
90
91      body = blocked.json()
92      assert body["error"] == "rate_limit_exceeded"
93      assert body["client_id"] == "ip:127.0.0.1# TestClient" # TestClient 주소
94
95
96  def test_check_validation_rejects_zero_cost(client):
97      # cost는 ge=1 → 0이면 422
98      r = client.post("/v1/check", json={"endpoint": "x", "cost": 0})
99      assert r.status_code == 422

```